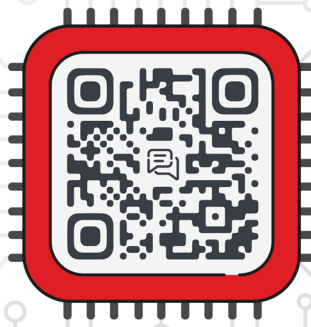




3



6



900 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

أنواع التعليمات (2)

د. مدحت الصوص

د. فراس ضعيف

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 30/11/2025

برمجة ا

عند كتابة كود بلغة C++ فإن المعالج لا يفهمه مباشرة لأنه لا يفهم سوى لغة الآلة

عند الضغط على Build في المحرر فإن الكود يمر بمرحلتين:

1. الترجمة (Compile)

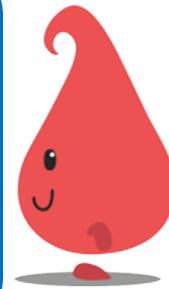
يقوم الـ Compiler بقراءة الكود والتحقق من سلامته اللغوية، ثم يقوم المترجم بترجمة الكود إلى لغة آلة جزئية غير مكتملة وغير جاهزة للتشغيل. ويُقصد بالتحقق من السلامة اللغوية للكود أنه في حال نسيان فاصلة منقوطة، أو قوس، أو أي قاعدة أخرى من قواعد اللغة، فإن المترجم سيبلغ بوجود خطأ قواعدي (Syntax Error).

2. الربط (Link)

يقوم الـ Linker بربط الملفات التي تحتوي على لغة آلة جزئية (الناتجة عن عملية الترجمة) معاً ومع المكتبات القياسية مثل مكتبة (iostream) لإنتاج ملف تنفيذي كامل. بعد أن تنتهي عملية الـ Build نحصل على برنامج جاهز للتنفيذ (Executable file (.exe) يحتوي على تعليمات بلغة الآلة يمكن للمعالج تنفيذها

برنامج مكتوب بالصيغة القياسية بلغة C++

```
#include <iostream>
using namespace std;
int main ()
{
    return 0;
}
```



الصيغة القياسية: هي الصيغة التي تلتزم بمعيار C++ الرسمي وهي تعمل في جميع بيئات ومترجمات C++ دون تعديل، ولا تعتمد على خصائص خاصة بمترجم أو نظام معين

شرح البرنامج السابق:

```
#include <iostream>
```

جميع التعليمات التي تبدأ بإشارة # تمثل تعليمات توجيهية للمترجم Compiler لتخبره ببعض التوجيهات التي يجب عليه القيام بها أي أن المترجم هو من يقوم بتنفيذها ولا تترجم إلى لغة الآلة كما هو حال التعليمات العادية، لا نضع فاصلة منقوطة بعد التعليمات التي تبدأ بإشارة # لأنها موجهة للمترجم وليست تعليمات عادية

```
using namespace std;
```

تستخدم هذه العملية لتضمين namespace المسمى std (standard) والذي يحتوي على العديد من التعليمات الجاهزة والموزعة على أكثر من مكتبة من ضمنها مكتبة iostream

```
int main ()
```

يقوم هذا السطر بتعريف التابع main حيث يبدأ تنفيذ أي برنامج مكتوب بلغة C++ من التعليمات الموجودة داخل التابع main داخل القوسين {} ولا يهم مكان التابع main سواء كان في أول البرنامج أو في آخره

```
return 0;
```

تقوم هذه التعليمة بإنهاء تنفيذ التابع main وإعادة القيمة 0 والتي تدل على عدم حدوث أي خطأ أثناء التنفيذ، لذلك فإنه يجب علينا كتابة التعليمات المراد تنفيذها في التابع main قبل تعليمة return

Escape characters

هناك بعض المحارف الخاصة التي نستطيع طباعتها باستخدام تعليمة Cout مثل إضافة سطر جديد أو Tab تبدأ جميع هذه المحارف ب "\" لتممييزها عن المحارف العادية فيما يلي جدول ببعض المحارف:

\n	يقوم بطباعة سطر جديد
\t	يقوم بطباعة tab والتي تساوي 8 فراغات
\\	يقوم بطباعة رمز \"
\"	يقوم بطباعة رمز "
\'	يقوم بطباعة رمز '



الثابت

ما الفائدة من تعريف الثابت؟

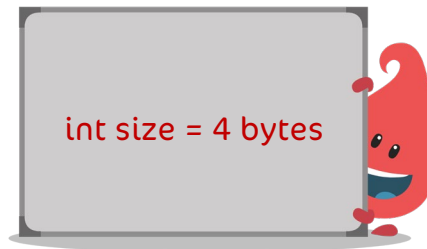
في بعض البرامج نحتاج لاستخدام أرقام أو محارف ثابتة مثل عدد الطلاب في مدرسة ما أو PI وإذا كنا بحاجة لاستخدام قيمة هذا الثابت عدة مرات في البرنامج فإننا سنحتاج لكتابة قيمة هذا الثابت بدقة عالية مرات عديدة وبذلك يجب علينا تعديل قيمته في جميع الأماكن ضمن البرنامج، عوضاً عن ذلك يمكننا تعريف قيمة الثابت مرة واحدة ومن ثم إعادة استخدامها كل ما احتجنا لها في البرنامج

`const int NumberOfStudents = 200;`

طريقة تعريف الثابت من نوع عدد صحيح:

شروط تسمية متحول او ثابت:

- 1- الانتباه إلى أن الحروف الكبيرة تختلف عن الحروف الصغيرة
- 2- لا يمكن أن نبدأ برقم في اسم المتحول يجب أن نبدأ بحرف
- 3- لا يمكن أن نستخدم رموز خاصة في اسم المتحول ويمكن استخدام Underscore(_) فقط



الشرط

متى نستخدم الشرط؟

عندما نريد تنفيذ خطوات معينة من خطوات الخوارزمية في حالة محددة وليس في كل الحالات هنا لا بد من ربط هذه الخطوات بشرط محدد حيث يتم تنفيذ هذه الخطوات فقط عند تحقق هذا الشرط ولا يتم تنفيذها أبداً إذا كان الشرط غير محقق

كيف نعبر عن الشرط بلغة الخوارزمية؟

إذا (شرط)
(مجموعة خطوات)

يكون (الشرط) عبارة عن صيغة منطقية والتي يكون ناتج تنفيذها إما true أو false بينما تتضمن (مجموعة الخطوات) أي عدد من الخطوات التي تنتمي لأنواع خطوات الخوارزمية الخمسة (قراءة، كتابة، إسناد، شرط، تكرار)

تتعلق قيمة الصيغة المنطقية بالقيم اللحظية للمتحويلات الموجودة ضمنها

مثال: اكتب خوارزمية تقوم بقراءة عدد وطباعة إذا كان موجب أو سالب

طريقة 1	طريقة 2 (الصيغة العامة)
اقرأ x	اقرأ x
إذا $(x \geq 0)$	إذا $(x \geq 0)$
اكتب "موجب"	اكتب "موجب"
إذا $(x < 0)$	وإلا
اكتب "سالب"	اكتب "سالب"
يجب ضمان تعاكس الشروط في الحالة السابقة لكيلا يقع البرنامج في الخطأ إن أخطأنا في الشرط	
بهذه الطريقة نكون قد ضمانا تعاكس الشروط بنسبة 100%	

ماذا لو أردنا تفحص ما إن كانت قيمة متغير x مساوية لقيمة ما ولنفرض 2

لا نكتب: إذا $(x = 2)$ لأن $(=)$ في لغة C++ تدل على الإسناد وليس على المساواة

بل نكتب: إذا $(x == 2)$ لأن $(==)$ في لغة C++ تدل على المساواة

أمثلة على شروط شائعة بلغة C++

$\text{if } (x > 2 \ \&\& \ y < 3)$	$\text{if } (x > 2 \ \ y < 3)$	$\text{if } (x != 2)$
تعني (إذا x أكبر من 2 و y أصغر من 3)	تعني (إذا x أكبر من 2 أو y أصغر من 3)	تعني (إذا x لا تساوي 2)

عندما قمنا بالانتقال إلى الصيغة العامة للشرط فقد قمنا بقسم فضاء الاحتمالات إلى شقين ولكن ماذا لو أردنا أن نقسم

فضاء الاحتمالات إلى 3 شقوق فأكثر

في هذه الحالة نقوم بإضافة شرط جديد مستقل ضمن (إلا) الخاصة بالشرط الأول

مثال: اكتب خوارزمية تقوم بقراءة عدد وطباعة ما إذا كان موجباً أو سالباً أو صفراً

طريقة 1	طريقة 2	طريقة 3 (التعميم)
اقرأ x	اقرأ x	اقرأ x
إذا $(x > 0)$ { اكتب "موجب" }	إذا $(x > 0)$ { اكتب "موجب" }	إذا $(x > 0)$ { اكتب "موجب" }
وإلا {	إذا $(x < 0)$ { اكتب "سالب" }	وإلا إذا $(x < 0)$ { اكتب "سالب" }
إذا $(x < 0)$ { اكتب "سالب" }	إذا $(x == 0)$ { اكتب " $x = 0$ " }	وإلا { اكتب " $x = 0$ " }
وإلا { اكتب " $x = 0$ " }		
}		

تتم عملية التنفيذ في الطريقة الثالثة على الشكل الآتي:

يقوم بالتحقق من صحة الشرط الأول إن لم يتحقق ينتقل إلى الثاني وإن لم يتحقق كلا الشرطين يقوم بتنفيذ ما داخل

(إلا) مباشرة

يمكننا عن طريق الطريقة 3 أن نقسم فضاء الاحتمالات لأي عدد نريد عن طريق إضافة (وإلا إذا)

مثال: اكتب خوارزمية لبرنامج يقرأ العدد المدخل من المستخدم ثم يخبره بعدد خاناته

حل 1 (في حالة الأعداد الموجبة)	حل 2
اقرأ x إذا $(x < 10)$ اكتب "خانة واحدة" إذا $(x < 100)$ و $x \geq 10$ اكتب "خانتان" إذا $(x < 1000)$ و $x \geq 100$ اكتب "3 خانات" إذا $(x > 1000)$ اكتب "4 خانات على الأقل"	اقرأ x إذا $(x < 10)$ و $x > -10$ اكتب "خانة واحدة" وإلا إذا $(x < 100)$ و $x > -100$ اكتب "خانتان" وإلا إذا $(x < 1000)$ و $x > -1000$ اكتب "3 خانات" وإلا اكتب "4 خانات على الأقل"
حل 3 (في حالة الأعداد الموجبة)	حل 4
اقرأ x إذا $(x < 0)$ اكتب "عدد سالب" وإلا { إذا $(x < 10)$ اكتب "خانة واحدة" وإلا إذا $(x < 100)$ اكتب "خانتان" وإلا إذا $(x < 1000)$ اكتب "3 خانات" وإلا اكتب "4 خانات على الأقل" }	اقرأ x إذا $(x < 0)$ $\{ x \leftarrow x * -1 \}$ إذا $(x < 10)$ اكتب "خانة واحدة" وإلا إذا $(x < 100)$ اكتب "خانتان" وإلا إذا $(x < 1000)$ اكتب "3 خانات" وإلا اكتب "4 خانات على الأقل"

حل 5: باستخدام مكتبة خاصة توفر خاصية القيمة المطلقة فيكون الحل مشابه للحل 4

كما لاحظنا لنستطيع التعامل مع الأعداد السالبة نضيف range له بداية ونهاية كما في الحل 2 ويمكننا من بداية البرنامج

تمييز الأعداد السالبة وتحويلها إلى موجبة كما في الحل رقم 4 و 5

1. Syntax error

هذا الخطأ يظهر لنا أثناء بناء البرنامج في مرحلة الترجمة مثل عدم وضع الفاصلة المنقوطة أو نسيان الأقواس أو تقديم خطوة على أخرى ككتابة cin لمتغير ما قبل حجز مكان له في الذاكرة

2. Runtime error

وهو يعرف بالخطأ التنفيذي ويحدث أثناء تشغيل البرنامج على عكس أخطاء الترجمة التي تحدث أثناء ال Build مثال: تعريف متغيرين x و y وإسناد قيمة 2 في y وقيمة 0 في x ثم طباعة ناتج قيمة y/x

3. Logical error

متعلق بفكرة الحل (الخوارزمية) حيث ينتهي البرنامج بشكل صحيح ولكن الجواب غير منطقي ولا يعلمنا ال Compiler بوجود الخطأ لأنه متعلق بالمنطق وليس بالكود البرمجي مثال: أن يخطئ المبرمج في كتابة خوارزمية قبل أن يحولها إلى كود برمجي البرنامج الصحيح هو البرنامج الذي يخلو من جميع أنواع الأخطاء المذكورة.



If there is no struggle,
there is no progress